

FungiForge — User Manual

A fungal ONT / Illumina / hybrid genomics pipeline (reads → assembly → annotation → identification → antifungal resistance → BGCs → One Health analysis)

Bright Adu

Genomics & Bioinformatics Core
Noguchi Memorial Institute for Medical Research
University of Ghana, Legon, Ghana

2026-07-30

Contents

1	1. Introduction	3
2	2. Concepts	4
3	3. Pipeline architecture	6
4	4. Installation	9
4.1	4.1 Local install (workstation / laptop)	9
4.2	4.2 Shared HPC install (SLURM + Apptainer/Singularity)	9
4.3	4.3 How FungiForge uses containers and conda envs	10
5	5. The fungiforge CLI and how to run	12
5.1	5.1 The console command	12
5.2	5.2 The <code>nextflow run</code> invocation and composable profiles	12
5.3	5.3 Quickstart (test fixture)	12
5.4	5.4 The test suite	13
6	6. Reference databases	15
6.1	6.1 The databases	15
6.2	6.2 Staging them	16
7	7. Inputs — the sample sheet	17
8	8. Configuration & parameters	18
9	9. The stages in detail	20
9.1	9.1 Stage 00 — Basecall (optional)	20
9.2	9.2 Stage 01 — Read QC & filter (mode-aware)	20
9.3	9.3 Stage 02 — Assembly (long-read) / Stage 02b — Assembly (short-read)	20
9.4	9.4 Stage 03a — ONT polish (Medaka), long-read isolates	20
9.5	9.5 Stage 03b — Short-read polish (Polypolish) / Illumina-only passthrough	21
9.6	9.6 Stage 04 — Decontamination + organelle split	21
9.7	9.7 Stage 05 — Assembly QC + completeness	21
9.8	9.8 Stage 06 — Repeat modelling + soft-masking	21
9.9	9.9 Stage 07 — Eukaryotic annotation (Funannotate)	22
9.10	9.10 Stage 08 — Identification (GCPSR)	22
9.11	9.11 Stage 09 — Antifungal resistance (the centerpiece)	22
9.12	9.12 Stage 10 — Mobile & repeat elements	22
9.13	9.13 Stage 11 — Biosynthetic gene clusters	22
9.14	9.14 Stage 12 — Novelty	23
9.15	9.15 Stage 13 — Eukaryote extras	23
9.16	9.16 Stage 14 — Report + master table	23
10	10. The antifungal-resistance module in depth	24

11	11. Outputs	26
11.1	11.1 Per-isolate outputs	26
11.2	11.2 The master table schema (the Layer-2 contract)	26
11.3	11.3 Provenance	27
12	12. Layer 2 — the One Health comparative analysis	29
13	13. Interpreting results	30
14	14. Troubleshooting	31
15	15. Reproducibility & provenance	32
16	16. Extending FungiForge	33
17	17. Glossary	34
18	18. Citation	35

1. Introduction

FungiForge is a Nextflow DSL2 pipeline that turns sequencing reads from a single fungal isolate — Oxford Nanopore (ONT) long reads, Illumina short reads, or both (hybrid) — into a de-novo assembly, a eukaryotic gene annotation, a species identification, an antifungal-resistance genotype, a mobile-element/mycovirus inventory, a biosynthetic-gene-cluster (BGC) catalogue, and a novelty verdict, then merges everything into one per-isolate report and a single row of a fixed-schema **master table**. A second, independent **R analysis layer** consumes those master rows across many isolates to build the air / human / surface **One Health** comparative story.

FungiForge is the **fungal sibling of the *forge* family** (CaptureForge designs capture panels, CallForge analyses targeted-capture germline callsets, MethylForge analyses methylomes). It follows the same engineering stance: **stand on validated upstream tools, add only the bespoke layers the science genuinely needs, and change inputs and configuration — never code — to move to a new organism, facility, or study.**

Its immediate use is the fungal arm of an airborne-AMR One Health study: culturing and sequencing fungi recovered from **air, human, and surface** compartments across abattoirs, clinics, markets and community settings, to ask which settings concentrate azole-resistant *Aspergillus fumigatus*, echinocandin-resistant *Candida/Nakaseomyces*, priority pathogens such as *Candida auris*, and candidate novel species or novel natural-product clusters. But the pipeline is **organism-agnostic**: any cultured fungal isolate with ONT and/or Illumina reads runs through it.

Who it is for. Genomics-core and bioinformatics staff running fungal isolate genomics who need a reproducible, container-pinned workflow that is honest about the two structural realities of fungal genomics: **(1)** ONT homopolymer indels fall exactly where antifungal-resistance point mutations live, so resistance calls on an ONT-only assembly are **provisional until hybrid-polished** (hybrid and Illumina-only assemblies are high-confidence, having no homopolymer-indel problem); and **(2)** there is **no ResFinder for fungi and no GTDB for fungi** — resistance and identification must be done with bespoke, curated logic rather than a single push-button database.

Status note used throughout this manual. FungiForge is implemented and validated **end-to-end on the synthetic test fixture** (all 15 stages wire and run under `-profile test`). Stage implementations are being exercised on a first real *A. fumigatus* isolate. Where a statement is a property of the *machinery* rather than a real biological result, the text says so. This manual documents **what the code actually does** and flags what is a placeholder or milestone-pending honestly (see §12–§14).

2. Concepts

This section covers the fungal-genomics background a FungiForge user needs to read the outputs correctly.

Three input modes — and why polishing matters for resistance. FungiForge assembles from whatever reads an isolate has, routed automatically by `meta.assembly_mode` (set from the samplesheet):

- **ONT-only (longread)** — Nanopore long reads span repeats and resolve a contiguous fungal genome (typically 30–50 Mb) that short reads alone fragment (Flye). Their weakness is **systematic error in homopolymer runs** — a run of six As may be read as five or seven — producing **false indels and frameshifts**. Antifungal-resistance calling reads single-residue changes in genes like *cyp51A*, *ERG11* and *FKS1*, so an uncorrected homopolymer error can *invent* a resistance mutation or *erase* a real one. **Stage 03a Medaka** (an ONT neural consensus) is always applied, but calls remain `provisional_ont_only`.
- **Hybrid (longread + Illumina)** — after Medaka, **Stage 03b Polypolish** uses the short reads to correct the residual indels Medaka cannot. This is the highest-quality assembly and yields **high-confidence** resistance calls.
- **Illumina-only (shortread)** — no ONT at all: **Stage 02b SPAdes** assembles directly from the short reads. There is no homopolymer-indel problem, so Medaka/Polypolish are skipped and calls are **high-confidence**. (The trade-off is contiguity: short-read-only assemblies are more fragmented than long-read ones, which can affect repeat-spanning features such as the *cyp51A* promoter TR locus.)

Every isolate carries a **polish mode** (`hybrid`, `illumina_only`, `ont_only`, or `hybrid_failed_ont_fallback`) and a **resistance-confidence flag**; ONT-only calls are labelled `provisional_ont_only` and never presented as clean.

ITS/UNITE barcoding and GCPSR multi-locus identification — no GTDB for fungi.

Bacteria have GTDB-Tk and a genome-taxonomy backbone; fungi do not. The universal fungal barcode is the **ITS region** (ITS1–5.8S–ITS2, between the SSU and LSU rRNA genes), classified against the **UNITE** reference database of curated species hypotheses (SH). ITS alone is often only genus-resolving, so FungiForge combines it with **genome-level average nucleotide identity (ANI)** (sourmash/skani vs type-strain sketches) and **MLST**, and reports a call under the principle of **GCPSR — Genealogical Concordance Phylogenetic Species Recognition**: a species is asserted with high confidence only when independent loci (ITS *and* genome ANI) agree. The result is always a species **plus a confidence and a method**, never a bare label. Guidance thresholds: ITS identity $\geq 98.5\%$ $\rightarrow$ species-level; 94–98.5% $\rightarrow$ genus/uncertain; $< 94\%$ $\rightarrow$ candidate novel.

Why antifungal resistance is bespoke: a curated panel + FungAMR references + a structural TR detector. No curated, sequence-bearing “ResFinder for fungi” exists. FungiForge builds resistance calling from three parts:

1. **A curated panel** (`af_resistance_panel.tsv`) — an interpretation overlay listing, per gene and per species, the drug class, the *mechanism* (substitution / loss-of-function / gain-of-function / over-expression / promoter tandem-repeat), the hotspot residue numbers, and the known resistance mutations, each with a literature source.

2. **FungAMR reference sequences** — FungAMR (Bédard/Landry *et al.*, *Nat Microbiol* 2025) ships a mutation *table* with no sequences; `bin/build_fungamr_refs.py` fetches the UniProt protein for every `ref_seq_uniprot_accession` and writes a species-labelled reference FASTA that the caller aligns the isolate’s proteins against, so hotspot residue numbering is anchored to the correct reference frame.
3. **A dedicated structural detector for the *A. fumigatus* cyp51A promoter tandem repeats.**

cyp51A TR34/TR46 promoter structure. The dominant *environmental* azole-resistance mechanism in *A. fumigatus* is **not** a point mutation but a **tandem repeat (TR) inserted into the cyp51A promoter** — a duplication of a ~34 bp (TR34) or ~46 bp (TR46) unit in the region between the two SrbA transcription-factor binding sites, which up-regulates the azole target. TR34 canonically pairs with the coding mutation **L98H**, and TR46 with **Y121F + T289A**. Because a TR is a structural insertion, it is invisible to any substitution caller; `bin/cyp51a_TR.py` locates cyp51A in the annotation, extracts the ~600 bp upstream promoter (strand-aware), and scans for a tandem array, reporting the unit length, copy number and TR type (TR34 / TR46 / TR46³ / TR53).

Biosynthetic gene clusters (BGCs). Fungi encode clustered secondary-metabolite pathways — non-ribosomal peptide synthetases (NRPS), polyketide synthases (PKS), terpene synthases, RiPPs, and hybrids — that produce mycotoxins, antibiotics and virulence factors. **fungiSMASH (antiSMASH 8, fungal mode)** detects them per isolate; whether a cluster is a known MIBiG compound or novel is resolved across all isolates by BiG-SCAPE in the Layer-2 comparative step.

Novelty by ANI/ITS distance. A candidate novel species is flagged when **genome ANI to the closest reference genome is <95%** (the strong signal, when a reference genome set is staged) **or**, as a fast fallback, when the **best ITS identity is <98.5%**. Both verdicts are reported with the GCPSR/polyphasic caveat — a genome distance is a hypothesis, confirmed only by multi-locus phylogeny and a formal polyphasic description.

3. Pipeline architecture

FungiForge Layer 1 runs **15 Nextflow stages** (modules/stage00...14), chained by subworkflows/fungiforge.nf and launched from main.nf. Each stage runs in its own pinned container (or conda env), publishes its outputs to results/<sample>/<NN>_<stage>/, and emits a small **result.json**; Stage 14 merges every stage's JSON into a per-isolate HTML report and one row of the master table. The diagram below is the authoritative map: solid edges are the assembly spine, dotted edges feed a downstream stage a specific artifact, dashed nodes are optional/skippable, and the amber cylinders are the reference databases staged under --data_dir.

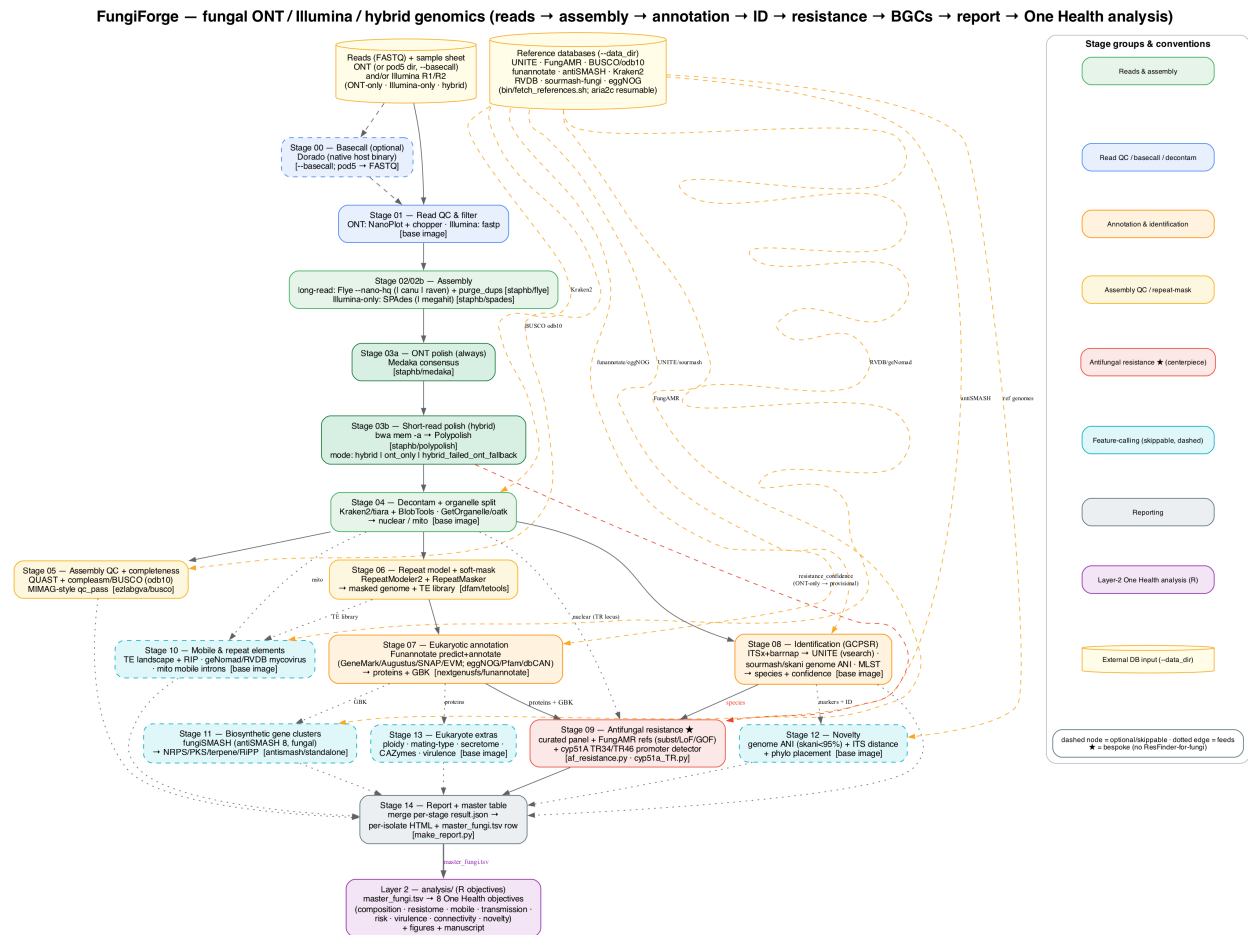


Figure 3.1: FungiForge Layer-1 pipeline DAG — the 15 stages colour-coded by group; the optional Dorado basecall and the four skippable feature-calling stages are dashed; external reference databases (amber) feed the ID/resistance/BGC/novelty stages; the antifungal-resistance stage (red, ★) is the bespoke centerpiece; everything converges on Stage 14, which emits **master_fungi.tsv** — the handoff to the Layer-2 One Health analysis.

Stage walkthrough (purpose · key tool/container · inputs → outputs).

#	Stage	Key tool(s) / container	Inputs → key output(s)
00	Basecall (<i>optional</i>)	Dorado (native host binary / base image)	pod5 dir → *.ont.fastq.gz
01	Read QC & filter	NanoPlot, chopper, fastp / base image	raw reads → *.ont.filt.fastq.gz, readqc.json (mode-aware)
02	Assembly (long-read)	Flye Canu Raven + purge_dups / staphb/flye	filtered ONT → *.assembly.fasta
02b	Assembly (short-read)	SPAdes MEGAHIT / staphb/spades	Illumina-only → *.assembly.fasta
03a	ONT polish	Medaka / staphb/medaka	draft + ONT → *.medaka.fasta (long-read isolates)
03b	Short-read polish (hybrid) / Illumina-only passthrough	bwa + Polypolish / staphb/polypolish	Medaka + Illumina → *.polished.fasta, polish.json; Illumina-only passes through (mode=illumina_only)
04	Decontam + organelle split	Kraken2 per contig (drops bacte- rial/archaeal/viral/human; records composition + fungal/non_fungal verdict) / base image	polished → *.nuclear.fasta, *.mito.fasta, decontam.json
05	Assembly QC + completeness	QUAST, compleasm/BUSCO / ezlabgva/busco	nuclear → assemblyqc.json (contiguity, BUSCO, qc_pass)
06	Repeat model + soft-mask	RepeatModeler2, RepeatMasker / dfam/tetools	nuclear → *.masked.fasta, *.telib.fasta, repeat.json
07	Eukaryotic annotation	Funannotate / nextgenusfs/funannotate	masked → *.proteins.faa, *.gbk, annotate.json
08	Identification (GCPSR)	ITSx, barrnap, vsearch/UNITE, sourmash/skani, MLST / base image	nuclear → *.species.txt, *.markers.fasta, identify.json
09	Antifungal resistance ★	af_resistance.py + cyp51a_TR.py / base image	proteins + species + nuclear + GBK → resistance.json
10	Mobile & repeat elements	te_summary.py, geNomad/RVDB / base image	TE library + mito → mobile.json
11	Biosynthetic gene clusters	fungiSMASH (antiSMASH 8) / antismash/standalone:8.0.0	GBK → bgc.json, *.regions.gbk
12	Novelty	skani + ITS distance + IQ-TREE / base image	nuclear + markers + ID → novelty.json
13	Eukaryote extras	nQuire/Smudgeplot, dbCAN, EffectorP, mating-type / base image	proteins + reads → extras.json

#	Stage	Key tool(s) / container	Inputs → key output(s)
14	Report + master table	<code>make_report.py</code> (jinja2) / base image	all <code>result.json</code> → <code>*.report.html</code> , <code>*.master.tsv</code>

Nextflow / profile model. `main.nf` parses the sample sheet and calls the single subworkflow FUNGIFORGE, which wires the 15 modules. Behaviour is set by **two composable profile groups** on the command line — an **execution** profile (`local` or `hpc_slurm`) plus a **packaging** profile (`conda`, `mamba`, `docker`, `singularity`, or `apptainer`). `conf/base.config` maps each stage’s resource **label** to its container image / conda env and its CPU/memory/time request; Nextflow picks the container when a container profile is active, otherwise the conda env. Heavy stages use their upstream community images; the light stages share the `aduton1000/fungiforge:0.1.0` base image.

The two-layer design and the master-table contract. Layer 1 is per-isolate genomics; **Layer 2 (analysis/)** is a set of R objective scripts that read only the master table. The handoff is a **fixed-schema TSV** so the two layers stay decoupled — Layer 2 can be re-run independently, and the schema deliberately mirrors the bacterial study’s `master_results` table so the fungal and bacterial One Health stories are directly comparable.

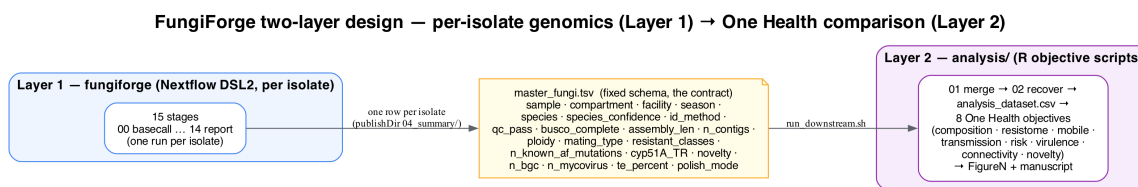


Figure 3.2: Two-layer architecture and the master-table handoff contract. Layer 1 (Nextflow) emits one `master_fungi.tsv` row per isolate to `results/04_summary/`; Layer 2 (R) merges those rows and runs eight One Health objectives.

4. Installation

FungiForge needs **Nextflow** (≥ 23.10) with **Java 17+**, plus either a container engine (Docker on a workstation, Apptainer/Singularity on HPC) or conda/mamba. You do **not** install the bioinformatics tools by hand — each stage declares its own pinned container or `env/*.yaml`, and Nextflow provisions it on first use.

4.1 4.1 Local install (workstation / laptop)

```
# 1. Nextflow (Java 17+ required)
curl -s https://get.nextflow.io | bash && sudo mv nextflow /usr/local/bin/
# or: mamba create -n nextflow -c bioconda 'nextflow>=23.10'

# 2. obtain FungiForge and enter the repository
cd /path/to/fungiforge

# 3. install the small Python front-end + bespoke callers (stdlib-light)
pip install -e .          # registers the `fungiforge` command; also usable in a
    ↪ conda env
```

macOS (Apple Silicon) — the amd64 emulation caveat. All heavy stage images are built **linux/amd64**; there are no **osx-arm64** builds for tools like Funannotate or antiSMASH. On Apple Silicon the docker profile therefore runs them under **Rosetta/QEMU emulation** — correct but slow. The pipeline is configured for this: `docker.runOptions` sets `--platform linux/amd64`, and heavy/slow options default **off** for local runs (`--run_interproscan false`, `--genome_id false`). Two consequences to plan around:

- Annotation (Stage 07) and BGC detection (Stage 11) can take **many hours** per isolate under emulation — budget accordingly, or run them on the HPC.
- **Dorado (Stage 00) runs as a native arm64 host binary, not in a container**, so basecalling stays fast on Apple Silicon. The light conda envs that have no **osx-arm64** build can be created with `CONDA_SUBDIR=osx-64 mamba env create -f env/<name>.yaml` (noted in `env/readqc.yaml`).

The container image. The light stages use `aduton1000/fungiforge:0.1.0`; build it once (native, no emulation) from the repo:

```
docker build -f env/Dockerfile -t aduton1000/fungiforge:0.1.0 .
```

4.2 4.2 Shared HPC install (SLURM + Apptainer/Singularity)

The production route is the cluster (native linux-64, no emulation). `docs/hpc_deployment.md` is the authoritative procedure; in brief:

```
# 1. code + tools (nextflow >=23.10, java 17+, apptainer or singularity)
git clone https://github.com/aduton1000/fungiforge && cd fungiforge
```

```

# 2. build the base image once (heavy public images pull automatically at run
    ↪ time)
apptainer build fungiforge.sif env/fungiforge.def          # or: singularity build ...

# 3. stage the reference databases once (hours; see §6)
export FUNGIFORGE_DB=/scratch/$USER/fungiforge_db
bash bin/fetch_references.sh

# 4. site config + Lmod module (the share/ wrappers)
cp share/fungiforge-env.sh.example ~/.fungiforge-env.sh  # edit paths /
    ↪ partition / account
module use /path/to/fungiforge/share/modulefiles
module load fungiforge

# 5. run (uses hpc_slurm, apptainer by default)
fungiforge-run --samplesheet samples.csv --run_interproscan true

```

The `hpc_slurm` profile (`conf/hpc_slurm.config`) sets the SLURM executor with time-based partition routing (short for ≤ 4 h tasks, long otherwise) unless you override `--slurm_partition` / `--slurm_account`; it caps concurrency at 100 queued jobs and rate-limits submission. The **share/ Lmod wrappers** — `fungiforge-run`, `fungiforge-env.sh.example`, and `modulefiles/fungiforge/0.1.0.lua` — put `FUNGIFORGE_HOME`, `FUNGIFORGE_DB`, the work dir, the profile, and the Apptainer bind paths (`APPTAINER_BINDPATH` must include the DB and work dirs) in one sourced site file so a run collapses to `fungiforge-run --samplesheet samples.csv`. Launch from a **writable** directory (Nextflow writes `.nextflow/` and `work/`) and put `work/` on scratch.

4.3 4.3 How FungiForge uses containers and conda envs

FungiForge deliberately uses **many small, isolated per-stage environments** rather than one monolith, because the stage toolchains conflict (a Flye/Medaka assembly stack, a Perl-heavy Funannotate stack, an antiSMASH stack, RepeatModeler’s `dfam/tetools` stack). Each stage’s `conf/base.config` label points at **both** a conda `env/*.yaml` and a container image; the active packaging profile selects which. Two container details are load-bearing and documented in the config:

- The `--data_dir` bind is explicit. Nextflow auto-mounts the project and work dirs but **not** `--data_dir`, so under Docker the profile injects `-v "$data_dir": "$data_dir"` (and under Apptainer/Singularity `-B "$data_dir"`) — otherwise the reference DBs are invisible to the containers. The path is quoted to survive spaces (e.g. a path containing spaces).
- The **antiSMASH image’s entrypoint is cleared** (`--entrypoint=""`) because `antismash/standalone` sets `ENTRYPOINT ["antismash"]`, which would hijack Nextflow’s bash launch; and the Docker profile passes `-e HOME=/tmp -e USER=fungiforge` so tools that call `getpass.getuser()/expanduser()` (Medaka/torch, Funannotate) do not crash on a host UID absent from the container’s `/etc/passwd`.

You never activate any of these by hand — Nextflow provisions the right env/container per process automatically.

Every image is pinned. `conf/base.config` names a versioned tag for each public image (`staphb/flye:2.9.6`, `staphb/spades:4.3.0`, `staphb/medaka:2.2.2`, `staphb/polypolish:0.6.1-bwa`, `ezlabgva/busco:v5.7.1_cv1`, `dfam/tetools:2.00`) and records, beside each, the manifest digest the tag resolved to when it was validated; `funannotate` has no versioned tag for the validated build, so it is pinned by digest (`nextgenusfs/funannotate@sha256:2eadfeb4...`). The `fungiforge` base image is built from `env/base.linux-64.lock`, an explicit URL+md5 lock of the whole solved environment (exported from the validated image with `bin/lock_env.sh`); `env/base.yml` carries the same versions as exact == pins for reading. Rebuilding from the lock reproduces the conda layer package-for-package (verified: two independent builds gave identical 453-package lists). `provenance.json` (§11.3) records what actually ran.

5. The fungiforge CLI and how to run

5.1 The console command

`pip install -e .` registers a small stdlib-only console command, **fungiforge** (defined in `fungiforge/cli.py`, entry point `fungiforge.cli:main`), with four subcommands:

- `fungiforge run ...` — thin wrapper over `nextflow run main.nf` (adds `--samplesheet`, `--data_dir`, `--outdir`, `-profile`, `-resume`; extra args pass through).
- `fungiforge samplesheet ...` — build a sample sheet by pairing ONT and/or Illumina read globs by sample id (§7).
- `fungiforge fetch-refs --data_dir DIR` — print the reference-DB fetch command (§6).
- `fungiforge version`.

The raw forms always work unchanged (`nextflow run main.nf ...`).

5.2 The nextflow run invocation and composable profiles

A run composes **one execution profile** and **one packaging profile**:

```
# single machine, Docker (on arm64 hosts: linux/amd64 emulated)
nextflow run main.nf -profile local,docker \
  --samplesheet samples.csv --data_dir /path/to/fungiforge_db
```

```
# SLURM cluster, native (no emulation)
nextflow run main.nf -profile hpc_slurm,singularity \
  --samplesheet samples.csv --data_dir /path/to/fungiforge_db
```

```
# equivalently, via the CLI front door
fungiforge run -profile local,docker \
  --samplesheet samples.csv --data_dir /path/to/fungiforge_db
```

- **Execution:** `local` (this single machine) or `hpc_slurm` (SLURM cluster).
- **Packaging:** `conda` | `mamba` | `docker` | `singularity` | `apptainer`.
- **test** is a third profile that points at the bundled synthetic fixture and is composed as `-profile test,docker` (usually with `-stub-run`) to validate the DAG in minutes.

`main.nf` enforces **fail-loud guards** at launch: it errors if `--samplesheet` is missing, warns if `--data_dir` is unset (identification / resistance / BGC / novelty need the reference DBs), and errors if `--basecall` is set without `--dorado_model`.

5.3 Quickstart (test fixture)

```
# validate all 15 stages wire end-to-end in minutes (machinery check, not data
  ↪ quality)
nextflow run main.nf -profile test,docker -stub-run
```

The `test` profile (`conf/test.config`) supplies a subsampled sample sheet, a mini reference DB, `busco_lineage = saccharomycetes_oddb10`, and writes to `results_test/`. Use it to confirm an install or a config change.

5.4 The test suite

Layer	What	How
Unit	every helper in <code>bin/*.py</code> and the CLI, on synthetic inputs (Kraken2 report fixture, GenBank records built in the test, synthetic proteins / promoters); the scripts are driven as subprocesses the way the pipeline drives them, with subprocess coverage of <code>bin/ + fungiforge/</code> ($\geq 85\%$ enforced)	<code>pip install -e "[dev]",</code> then <code>COVERAGE_PROCESS_START=.coveragerc</code> <code>coverage run -m pytest</code> <code>test/unit && coverage</code> <code>combine && coverage</code> <code>report</code>
Static	<code>nextflow lint</code> at zero warnings (strict syntax); <code>shellcheck</code> at warning level on every shell script	<code>nextflow lint .;</code> <code>shellcheck -S warning -x</code> <code>bin/*.sh share/bin/*</code>
DAG	the 15 stages on the three-mode fixture under <code>-stub-run</code> (routing, joins, DB_CHECK gate, provenance)	<code>nextflow run main.nf</code> <code>-profile local,test</code> <code>-stub-run</code>
nf-test	<code>parse_row/resolve_path</code> routing cases (ONT-only, hybrid, Illumina-only, unpaired refused), one stub test per module (every declared output emitted once, named after the sample), the whole DAG with and without the skip flags	<code>nf-test test</code> (<code>nf-test.config</code> , cases under <code>test/nf-test/</code>)
Containers	the fixture through the real images with Docker: DB_CHECK and READ_QC complete in-container, versions are recorded, and the assemblers' failure on the synthetic reads is recorded by the status contract, not hidden	<code>bash</code> <code>test/run_docker_fixture.sh</code>

Layer	What	How
Real data	CEA10 hybrid, DF-005 Illumina-only, C87 TR34/L98H on the development deployment	<code>docs/UPGRADE_PLAN.md</code> (W0.4, W1.1)

The first four layers run on every push (`.github/workflows/ci.yml`); the last two need the image set and databases and run locally or on the development deployment.

6. Reference databases

FungiForge follows the forge convention: **fetch and checksum every database up front, never mid-run**. `bin/fetch_references.sh` stages them all into one `--data_dir` root using **aria2c** (16 parallel connections, `--continue` to resume a broken transfer, infinite retries; `curl -C -` fallback). Each database is guarded by a `.done` sentinel and recorded in `MANIFEST.tsv`; **one database failing does not abort the others**, and version-sensitive URLs (UNITE, Kraken2, sourmash-fungi, RVDB) resolve the current release at run time and log-and-skip rather than guess on a 404. Databases total roughly **150–250 GB**, so they are staged on an external/scratch drive.

6.1 6.1 The databases

Database	Provides	Stage(s)	<code>fetch_references.sh</code> step
Container images	all stage images (Flye, Medaka, antiSMASH 8, Funannotate, dfam/tetools, BUSCO)	all	images
antiSMASH databases (~9 GB)	fungiSMASH BGC detection	11	antismash (via the antiSMASH 8 image)
Funannotate DB (~30–50 GB: Pfam, dbCAN, MEROPS, InterPro, BUSCO)	eukaryotic annotation	07	funannotate (<code>funannotate setup -i all</code>)
eggNOG (~50 GB)	functional annotation	07	egglog (<code>download_egglog_data.py</code>)
BUSCO / compleasm <code>fungi_odb10</code> (+ order lineages)	assembly completeness	05	busco
UNITE general FASTA (Fungi v10.0, 2025)	ITS species identification	08	unite
Kraken2 (PlusPF-8 GB)	decontamination	04	kraken2

Database	Provides	Stage(s)	<code>fetch_references.sh</code> step
sourmash/RefSeq-fungi (GenBank fungi k=31)	genome-ANI ID & novelty	08, 12	<code>refseq_fungi</code>
FungAMR tables + built reference proteins	antifungal-resistance calling	09	<code>fungamr</code> (+ <code>build_fungamr_refs.py</code>)
RVDB-prot (v31.0 RdRp)	mycovirus / EVE screen (staged now; consumed from the W2.8 mobile/mycovirus upgrade)	10	<code>rvdb</code>

Checked before every run. The first task of a run, `DB_CHECK`, verifies that every database the enabled stages need is present and complete (the `.done` marker `fetch_references.sh` writes plus the key files each stage opens: UNITE FASTA, Kraken2 `*.k2d`, BUSCO lineage, funannotate Pfam-A.hmm, antiSMASH `clusterblast/`, FungAMR tables + reference proteins; `refseq_fungi` only with `--genome_id`; Kraken2/antiSMASH only when their stages are on). Anything missing is listed once and the run stops in seconds instead of hours later; `--allow_missing_db` turns that into a warning, and the dependent stages then record `skipped`. `--funannotate_db` / `--antismash_db` overrides are honoured. The findings are written to `pipeline_info/db_manifest.json` (with each database's `MANIFEST.tsv` source line) and embedded in `provenance.json`.

6.2 6.2 Staging them

```
export FUNGIFORGE_DB="/path/to/fungiforge_db"
bash bin/fetch_references.sh                # all steps (many hours;
↪ resumable)
bash bin/fetch_references.sh images antismash fungamr # only selected steps
```

Re-running is safe and resumes where it stopped. The FungAMR step is special: FungAMR ships a **mutation table with no sequences**, so after downloading the tables the script calls `build_fungamr_refs.py`, which reads every `ref_seq_uniprot_accession`, fetches those proteins from **UniProt** (batched, with recursive batch-splitting so one obsolete accession never sinks its neighbours), and writes `fungamr/reference_proteins.faa` with `>GENE__ACC__Species` headers — the sequences Stage 09 aligns against. **MARDy is not fetched** (its REST API is API-key-gated and its content is largely subsumed by FungAMR).

Note on refreshing version-sensitive URLs. The script embeds the current default URL and the recipe to resolve a newer one for UNITE (PlutoF DOI), Kraken2 (genome-idx S3), sourmash-fungi (UC Davis prepared DB) and RVDB (Institut Pasteur). If a URL 404s, that step logs `FAILED` in the manifest and continues; resolve the new link and re-run just that step.

7. Inputs — the sample sheet

The only per-run input is a **CSV sample sheet** (`--samplesheet`), with this exact schema (parsed in `main.nf`):

```
sample,ont_fastq,illumina_r1,illumina_r2,compartment,facility,season
```

- **sample** — unique isolate id (becomes the output directory and every filename).
- **ont_fastq** — the ONT FASTQ. **Optional** — required only when the isolate has no Illumina reads. When `--basecall` is set this column is instead the **pod5 directory** Dorado basecalls.
- **illumina_r1, illumina_r2** — paired short reads. **Optional** individually, but they come as a pair (one without the other is an error). With ONT present → the Stage 03b hybrid polish runs (under `--hybrid auto`). Without ONT → the isolate is **Illumina-only** and is assembled by SPAdes (Stage 02b). Missing reads are substituted with the `assets/NO_ONT / NO_R1 / NO_R2` sentinels so the channel plumbing stays well-typed.
- **compartment, facility, season** — the One Health metadata carried untouched to the master table and consumed by Layer 2 (e.g. `AIR / HUMAN / SURFACE`; a facility label; a season). Default to `NA` when absent.

Each row must provide either `ont_fastq` or the `illumina_r1+illumina_r2` pair (or all three). The mode is auto-detected per row (`meta.assembly_mode`): ONT present → `longread` (ONT-only or hybrid); ONT absent → `shortread` (Illumina-only).

A generic example mixing all three modes:

```
sample,ont_fastq,illumina_r1,illumina_r2,compartment,facility,season
```

```
|
|  ↪  AF_AIR_001,reads/AF_AIR_001.ont.fastq.gz,reads/AF_AIR_001_R1.fastq.gz,reads/AF_AIR_001_R2.fastq.gz,NA,NA,NA
|  ↪  CA_HUM_014,reads/CA_HUM_014.ont.fastq.gz,,,HUMAN,Clinic_2,Wet
|
|  ↪  CA_SUR_009,,,reads/CA_SUR_009_R1.fastq.gz,reads/CA_SUR_009_R2.fastq.gz,SURFACE,Clinic_2,Wet
```

Here `AF_AIR_001` is **hybrid** (ONT + Illumina → hybrid-polished, high-confidence resistance), `CA_HUM_014` is **ONT-only** (provisional resistance), and `CA_SUR_009` is **Illumina-only** (SPAdes assembly, high-confidence resistance).

Building one automatically. `fungiforge samplesheet` pairs ONT and/or Illumina globs by sample id and stamps the metadata (omit `--ont` entirely for an Illumina-only cohort):

```
fungiforge samplesheet \
  --ont 'reads/*.ont.fastq.gz' \
  --illumina-r1 'reads/*_R1*.fastq.gz' --illumina-r2 'reads/*_R2*.fastq.gz' \
  --compartment AIR --facility Abattoir_1 --season Dry \
  -o samples.csv
```

It derives the id by stripping the read suffix, matches R1/R2 to the ONT id (dropping `_R1/_1` markers), and leaves the Illumina columns blank for any sample without short reads. Edit the CSV afterwards to correct per-sample compartment/facility/season if they differ.

8. Configuration & parameters

Set parameters with `--key value` on the command line (there is no `params-file` convention baked in, but `-params-file x.yaml` works as usual for Nextflow). Compute caps and stage toggles are the knobs you will touch most.

Key parameters (real defaults from `nextflow.config`).

Param	Default	Meaning
<code>--samplesheet</code>	<code>null</code>	required — the CSV sample sheet
<code>--data_dir</code>	<code>null</code>	reference-database root (§6); warns if unset
<code>--outdir</code>	<code>results</code>	results directory
<code>--hybrid</code>	<code>auto</code>	ONT isolates only: auto (polish with Illumina if present) on off
<code>--assembler</code>	<code>flye</code>	long-read assembler (ONT/hybrid): flye canu raven
<code>--sr_assembler</code>	<code>spades</code>	short-read assembler (Illumina-only): spades megahit
<code>--purge_dups</code>	<code>true</code>	collapse heterozygous haplotigs after long-read assembly (skipped for short-read)
<code>--min_contig_len</code>	<code>500</code>	drop contigs shorter than this, or with fewer than 4 distinct bases, at short-read assembly and before annotation (funannotate sort --minlen); short-read assemblers leave many tiny fragments and occasional homopolymer stubs that funannotate rejects
<code>--ont_min_qual</code>	<code>10</code>	chopper Q filter (Stage 01)
<code>--ont_min_len</code>	<code>1000</code>	chopper length filter (Stage 01)
<code>--basecall</code>	<code>false</code>	run Dorado on a pod5 dir (<code>ont_fastq</code> column = pod5 dir)
<code>--dorado_model</code>	<code>r1041_e82_400bps_sup_v5.2.0</code>	Dorado model; also Medaka's model
<code>--dorado_duplex</code>	<code>false</code>	Dorado duplex basecalling

Param	Default	Meaning
<code>--busco_lineage</code>	auto	auto (order-specific after ID) <code>fungi_odb10</code> <code><lineage></code>
<code>--genome_id</code>	false	Stage 08 genome-level sourmash gather (slow emulated; ITS is primary)
<code>--run_interproscan</code>	false	Stage 07 InterProScan (heavy; default off, enable on a cluster)
<code>--ploidy</code>	auto	ploidy handling
<code>--genemark_key</code>	null	path to a free-academic GeneMark license <code>.gm_key</code>
<code>--af_panel</code>	bundled <code>af_resistance_panel.tsv</code>	curated resistance panel (Stage 09)
<code>--skip_decontam</code>	false	skip Stage 04 decontamination
<code>--skip_mge</code>	false	skip Stage 10 mobile elements
<code>--skip_bgc</code>	false	skip Stage 11 BGCs
<code>--skip_novelty</code>	false	skip Stage 12 novelty
<code>--skip_extras</code>	false	skip Stage 13 extras
<code>--max_cpus</code>	16	max cores per task on the local machine
<code>--max_memory</code>	120.GB	memory ceiling
<code>--max_time</code>	96.h	time ceiling (fungal annotation is slow, esp. emulated)
<code>--slurm_partition /</code> <code>--slurm_account</code>	null	SLURM routing (else time-based <code>short/long</code>)

Stage toggles genuinely skip. A `--skip_*` flag makes the subworkflow emit an *empty channel* for that stage rather than running the process emptily — so the stage’s container is never even pulled. Skippable stages are 10 (mobile), 11 (BGC), 12 (novelty), 13 (extras), plus 04 decontam via `--skip_decontam`. The assembly-through-report spine (00–09, 14) is not skippable.

Per-process resources. Resources are declared per stage via labels in `conf/base.config` (e.g. `assembly` gets up to 12 CPUs / 64 GB / 24 h; `annotate` up to 12 CPUs / 64 GB / 72 h; `repeat` up to 12 CPUs / 48 GB / 48 h). CPU-bound tools receive `task.cpus`. The default process policy retries on out-of-memory/time exit codes (137/139/140/143/247) up to twice, scaling memory and time with the attempt number.

9. The stages in detail

Each stage publishes to `results/<sample>/<NN>_<stage>/` and emits a `result.json`. Generic example commands below show the pipeline invocation that exercises the stage; you normally run the whole pipeline, not stages individually.

9.1 9.1 Stage 00 — Basecall (optional)

Runs **Dorado** on a pod5 directory to produce the ONT FASTQ, only when `--basecall` is set (and `--dorado_model` must be given, or `main.nf` errors). Dorado runs as a **native host binary**, which keeps basecalling fast on Apple Silicon. Supports duplex via `--dorado_duplex`.

```
nextflow run main.nf -profile local,docker --samplesheet pod5_samples.csv \
  --data_dir /path/to/fungiforge_db --basecall --dorado_model
  ↪ r1041_e82_400bps_sup_v5.2.0
```

9.2 9.2 Stage 01 — Read QC & filter (mode-aware)

For **long-read isolates** (ONT present): **NanoPlot** on raw and filtered reads; **chopper** filters by quality (`--ont_min_qual`, default Q10) and length (`--ont_min_len`, default 1000 bp); **Illumina (if present)**: **fastp**. For **Illumina-only isolates**: **fastp** only, and an empty ONT placeholder is emitted to keep the tuple contract. `readqc.json` records the platform (`ont | hybrid | illumina`) and the filtered ONT FASTQ (long-read) is carried forward.

9.3 9.3 Stage 02 — Assembly (long-read) / Stage 02b — Assembly (short-read)

Long-read isolates (`assembly_mode == longread`) are assembled with the chosen `--assembler`: **Flye** `--nano-hq` (default), **Raven**, or **Canu** (`genomeSize=35m`). When `--purge_dups true` (default), `purge_dups` collapses heterozygous haplotigs via a minimap2 self-alignment; if `purge_dups` is unavailable the raw assembly is kept.

Illumina-only isolates (`assembly_mode == shortread`) are assembled by **Stage 02b** with `--sr_assembler`: **SPAdes** `--isolate` (default; built-in read error-correction) or **MEGAHIT**. `purge_dups` (a long-read haplotig step) does not apply. Both stages emit the same `*.assembly.fasta` so the two paths merge transparently downstream.

9.4 9.4 Stage 03a — ONT polish (Medaka), long-read isolates

Medaka (`staphb/medaka`) computes a neural consensus of the draft using the ONT reads and the `--dorado_model`. If Medaka produces no consensus the draft passes through unchanged. Emits `*.medaka.fasta`. Runs only for long-read isolates (Illumina-only assemblies skip it — there is no ONT and no homopolymer error to correct). Medaka and Polypolish live in different containers, which is why the polish is split into 03a and 03b.

9.5 9.5 Stage 03b — Short-read polish (Polypolish) / Illumina-only passthrough

For **hybrid isolates** (ONT + Illumina, `--hybrid auto/on`), this stage polishes the Medaka consensus with **Polypolish** (`staphb/polypolish`). Polypolish requires **all** alignments per read (`bwa mem -a`) plus an insert-size filter — the module runs `bwa index`, two `bwa mem -a` passes, `polypolish filter`, then `polypolish polish`. For **Illumina-only isolates** the SPAdes assembly is already short-read-derived, so it passes through unchanged. The reported **mode is honest**:

- **hybrid** — the short-read pass succeeded (`resistance_confidence: high`);
- **illumina_only** — Illumina-only assembly, no ONT (`resistance_confidence: high`; no homopolymer-indel risk);
- **ont_only** — long-read isolate with no Illumina reads (`provisional_ont_only`);
- **hybrid_failed_ont_fallback** — the pass was attempted but failed, so the Medaka consensus is kept and the mode says so (`provisional_ont_only`) — never silently claimed as a clean hybrid.

The `polish.json` records the mode, the polisher version, the bp changed, and the `resistance_confidence` that rides downstream to flag Stage 09 calls.

9.6 9.6 Stage 04 — Decontamination + organelle split

Kraken2 (against `--data_dir/kraken2`) plus tiara/BlobTools-style GC×coverage triage drop bacterial/human contigs, keeping fungal + unclassified contigs as the **nuclear** assembly; the **mitochondrial** genome is separated (`GetOrganelle/oatk`) because its mobile introns matter in Stage 10. Skipped (assembly passes through as nuclear) when `--skip_decontam` or no `--data_dir`. Emits `*.nuclear.fasta`, `*.mito.fasta`, `decontam.json`.

Honest limitation. The shipped module keeps *all* contigs as nuclear (a `seqkit` pass) and writes an empty mito placeholder — the BlobTools refinement and organelle extraction are wired for milestone 3. Kraken2 is invoked but its classification is not yet used to drop contigs.

9.7 9.7 Stage 05 — Assembly QC + completeness

QUAST contiguity plus **compleasm** (preferred) or **BUSCO** completeness against the order-specific ODB10 lineage (`fungi_odb10` fallback under `--busco_lineage auto`). `bin/assembly_qc.py` computes assembly length, contig count, N50, largest contig and the BUSCO % complete, and sets a **MIMAG-style qc_pass** flag (true when the assembly is non-empty and BUSCO ≥80% or unavailable). Emits `assemblyqc.json`.

9.8 9.8 Stage 06 — Repeat modelling + soft-masking

RepeatModeler2 builds a de-novo TE library (`-LTRStruct`), **RepeatMasker** soft-masks the nuclear genome (`-xsmall`) — both from the `dfam/tetools` image. Emits the soft-masked genome (for annotation), the **TE library** (for Stage 10), and `repeat.json`.

9.9 9.9 Stage 07 — Eukaryotic annotation (Funannotate)

Funannotate **predict** (GeneMark / Augustus / SNAP / GlimmerHMM combined by EVM) then **annotate** (eggNOG, Pfam, dbCAN, MEROPS; InterProScan only with `--run_interproscan true`), from the `nextgenusfs/funannotate` image with `FUNANNOTATE_DB=--data_dir/funannotate`. A free-academic GeneMark key can be supplied via `--genemark_key`. Emits the **proteins FASTA** and the **GenBank (.gbk)** that Stages 09 and 11 consume, plus `annotate.json`. This is the slowest stage under emulation.

9.10 9.10 Stage 08 — Identification (GCPSR)

barrnap finds the rRNA operon; `extract_rrna_region.py` extracts a padded window around it so **ITSx** stays under the HMMER 100 kb limit (it aborts on whole chromosomes); **ITSx** carves ITS1/5.8S/ITS2; **vsearch** classifies ITS against **UNITE** (`--id 0.90`, top hits). Optionally (`--genome_id true`, slow emulated) **sourmash gather** does genome-level ANI. `id_classify.py` resolves a species call with confidence and method under the GCPSR logic (§2): ITS $\geq 98.5\%$ $\rightarrow$ high/species; 94–98.5% $\rightarrow$ medium; genus-only $\rightarrow$ low; genome-ANI concordance with ITS $\rightarrow$ high (GCPSR). Emits `*.species.txt`, `*.markers.fasta`, `identify.json` (including the top ITS identity, reused by novelty).

9.11 9.11 Stage 09 — Antifungal resistance (the centerpiece)

The bespoke core; see §10 for full depth. `af_resistance.py` takes the Funannotate proteins, the species call, the nuclear assembly and the GBK; selects species-relevant panel rows; finds the isolate ortholog of each target gene; reads residues at the panel hotspots; classifies each as a known resistance mutation, novel hotspot variant, wild-type, loss-of-function, gain-of-function, or over-expression target; and runs the `cyp51a_TR.py` promoter-TR detector for *A. fumigatus*. Every call carries a confidence flag tied to the polish mode. Emits `resistance.json`.

9.12 9.12 Stage 10 — Mobile & repeat elements

`te_summary.py` tallies the TE library by superfamily (LTR Gypsy/Copia, TIR, LINE, Helitron); **geNomad** (with `--data_dir/genomad_db`) scans for mycovirus / endogenous viral elements; `mobile_merge.py` combines them, noting the **DNA-only caveat** (RNA mycoviruses are not captured by WGS). Skippable with `--skip_mge`. Emits `mobile.json`.

9.13 9.13 Stage 11 — Biosynthetic gene clusters

fungiSMASH (antiSMASH 8, fungal taxon) detects NRPS/PKS/terpene/RiPP/hybrid clusters from the GBK; `bgc_summary.py` counts clusters by product type. MIBiG-known-vs-novel and BiG-SCAPE networks are resolved across isolates in Layer 2. Skippable with `--skip_bgc`. Emits `bgc.json` and the region GBKs.

9.14 9.14 Stage 12 — Novelty

`novelty_call.py` combines **skani genome ANI** (<95% → candidate novel; the strong signal when a reference genome set is staged under `refseq_fungi_genomes/`) with the **ITS distance** fallback (<98.5% → candidate novel), and reports the verdict with the GCPSR/polyphasic caveat. Skippable with `--skip_novelty`. Emits `novelty.json`.

9.15 9.15 Stage 13 — Eukaryote extras

`extras.py` derives the **mating-type idiomorph** (MAT1-1 / MAT1-2 from protein names) and tallies secretome/CAZyme/virulence keyword hits from the annotation; ploidy/heterozygosity (nQuire / Smudgeplot / GenomeScope2) and richer secretome tools (SignalP6 / DeepTMHMM / EffectorP3) are wired to run when reads and models are available. Skippable with `--skip_extras`. Emits `extras.json`.

9.16 9.16 Stage 14 — Report + master table

`make_report.py` merges every per-stage `result.json` into a **self-contained per-isolate HTML report** and appends the isolate's **master row** to `results/04_summary/<sample>.master.tsv` (published to both the sample's `14_report/` dir and the shared `04_summary/`). Missing stages become **NA** — never a hard failure. The master schema is the Layer-2 contract (§11).

10. The antifungal-resistance module in depth

This is where FungiForge adds the most value, because there is **no ResFinder for fungi**. The flow below is `bin/af_resistance.py` (calling `bin/cyp51a_TR.py`), driven by the curated panel and the FungAMR references.

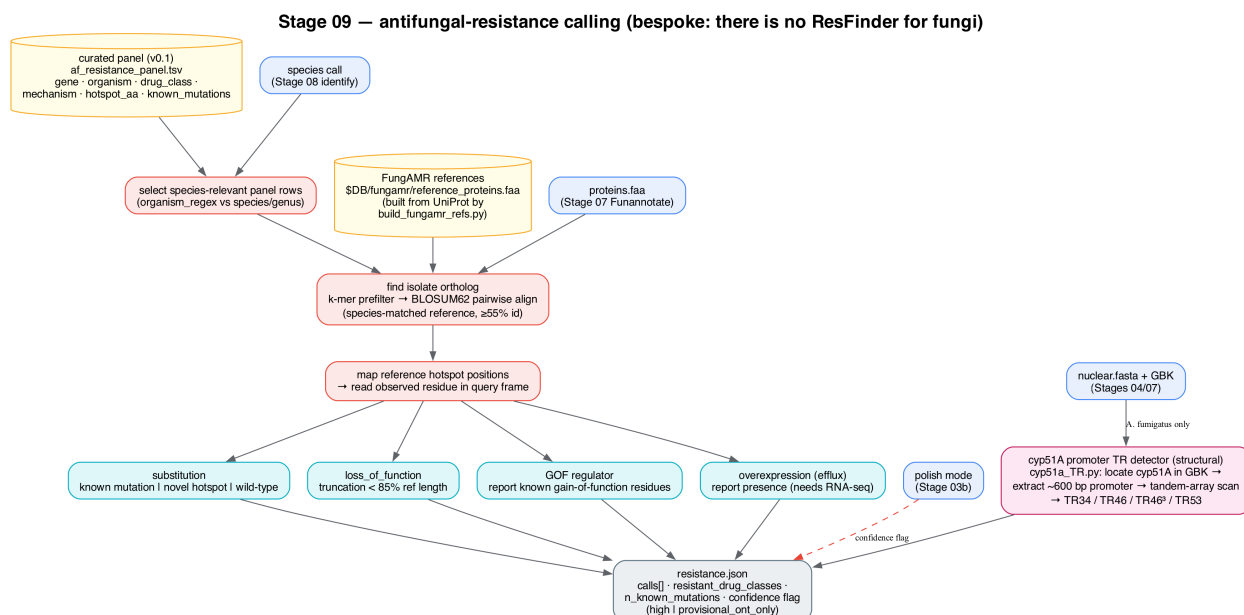


Figure 10.1: Stage 09 antifungal-resistance calling flow: species-relevant panel rows drive ortholog detection against species-matched FungAMR references; observed residues at panel hotspots are classified by mechanism; the *cyp51A* promoter TR detector runs structurally for *A. fumigatus*; the polish mode sets the confidence flag.

1 — The curated panel (`af_resistance_panel.tsv`). A tab-separated interpretation overlay, one row per gene×species, with columns `gene`, `organism_regex`, `drug_class`, `drugs`, `mechanism`, `hotspot_aa`, `known_mutations`, `note`, `source`. It covers the clinically important axes:

- **Azoles:** *A. fumigatus* **cyp51A** (G54/L98H/Y121F/G138C/M220/T289A/G448S...), **cyp51A promoter** (TR34/TR46/TR46³/TR53), **hmg1**, *cyp51B*; *Candida albicans* / *auris* / *parapsilosis* / *tropicalis* / *Nakaseomyces glabratus* / *Cryptococcus neoformans* **ERG11** (Y132F, K143R, F126L...); the azole efflux regulators **TAC1/MRR1/UPC2/PDR1** (GOF) and pumps **CDR1/CDR2/MDR1/AtrF** (over-expression).
- **Echinocandins:** **FKS1** HS1/HS2 (S645P, F641S...) and **FKS2** across *Candida*/*Nakaseomyces* and *A. fumigatus* (S678P).
- **Pyrimidines (5-FC):** **FUR1** (R101C) and loss-of-function of **FCY1/FCY2**.
- **Polynes:** loss-of-function of **ERG3/ERG6/ERG2**.

2 — Species-relevance filter. Only rows whose `organism_regex` matches the isolate's species (or genus, or spp.) are searched — so *A. fumigatus* isolates are not mis-tested against *Candida*

ERG11 numbering.

3 — Ortholog detection and residue mapping. For each relevant gene the caller pulls the matching **FungAMR reference from the isolate’s own species** (headers are `GENE__ACC__Species`; species-matched, else genus, else longest), because the panel’s hotspot numbering is species-specific and a wrong-species reference would misnumber every residue. It finds the isolate ortholog among the Funannotate proteins with a **k-mer prefilter** → **BLOSUM62 global pairwise alignment** ($\geq 55\%$ identity), then reads the residue in the isolate frame aligned to each 1-based reference hotspot.

4 — Mechanism-aware calling. Each panel row is classified by its mechanism:

- **substitution** — report `known_resistance_mutation` (residue matches a known mutant), `novel_hotspot_variant` (changed but not a known mutant), or `wild_type`.
- **loss_of_function** — flag a candidate LoF when the ortholog is truncated ($< 85\%$ of reference length).
- **GOF** (regulators) — report gene presence plus any known gain-of-function residues (confirm with expression/phenotype).
- **overexpression** (efflux) — report presence only; over-expression needs RNA-seq and is not callable from a genome.

5 — The cyp51A promoter TR detector (`cyp51a_TR.py`). For *A. fumigatus* with an assembly, the structural detector locates cyp51A in the GBK, extracts the ~600 bp upstream promoter (strand-aware), scans for a tandem array (unit lengths 20–60 bp, $\geq 90\%$ identity between copies, ≥ 2 copies), and snaps the unit length to the nearest canonical TR (TR34/TR46/TR53; TR46³ when ≥ 3 copies of the ~46 bp unit). A detected TR is emitted as a high/medium-confidence azole-resistance call annotated with its canonical coding-mutation pairing (TR34+L98H, TR46+Y121F/T289A).

6 — The confidence flag (provisional vs high). Every call carries a confidence derived from the polish mode: **high** for a hybrid-polished assembly, **provisional_ont_only** for ONT-only. This is the pipeline’s central honesty guarantee — an ONT-only substitution or frameshift call could be a homopolymer artefact, so it is never presented as confirmed. The promoter TR call (a structural signal) is high-confidence when hybrid and medium otherwise.

7 — Graceful degradation. Without staged FungAMR references the gene is reported as `no_reference` (“searched but unresolved”) rather than guessed, and the structural TR scan still runs. The output `resistance.json` summarises the resistant drug classes, the count of known mutations, and whether a reference was available.

Interpretation caveat (in the panel header and every report). These are **genotype** calls that flag *known* resistance-associated changes. **MIC / AFST phenotype confirmation is always required**, and over-expression/GOF mechanisms need expression data.

11. Outputs

11.1 11.1 Per-isolate outputs

Under `results/<sample>/`:

Output	Location	Notes
Filtered reads	01_readqc/	ONT filtered FASTQ + NanoPlot
Polished assembly	03_polish/	*.polished.fasta + polish.json (mode)
Nuclear / mito	04_decontam/	*.nuclear.fasta, *.mito.fasta
Assembly QC	05_assembly_qc/	assemblyqc.json (N50, BUSCO, qc_pass)
Annotation	07_annotate/	*.proteins.faa, *.gbk
Identification	08_identify/	*.species.txt, identify.json
Resistance	09_resistance/	resistance.json (calls + confidence + TR)
Mobile / BGC / novelty / extras	10_...-13_...	per-feature result.json
Per-isolate report	14_report/	*.report.html (self-contained)
Master row	14_report/ and 04_summary/	*.master.tsv

11.2 11.2 The master table schema (the Layer-2 contract)

`make_report.py` writes a fixed, append-only column set to `results/04_summary/<sample>.master.tsv`:

```
sample, compartment, facility, season,  
species, species_confidence, id_method,  
qc_pass, busco_complete, busco_lineage, assembly_len, n_contigs,  
ploidy, mating_type,  
resistant_classes, n_known_af_mutations, cyp51A_TR,  
novelty, n_bgc, n_mycovirus, te_percent,  
polish_mode,  
sample_verdict, contam_removed_pct, top_taxon,  
stages_failed
```

11.2.1 Stage status contract

Every per-stage JSON (`results/<sample>/NN_<stage>/<sample>.<stage>.json`) carries a `status` and a `tools` block, written by `bin/ff_status.sh` / `bin/ff_status.py` at the end of the task:

status	Meaning
ok	every tool that ran exited 0

status	Meaning
partial	an optional step failed and the documented fallback was used (e.g. Polypolish failed, the medaka assembly was carried forward, resistance confidence dropped to provisional)
failed	a required tool failed. The task exits non-zero and the run stops after pending tasks, except for the best-effort stages (BGC, extras), which record failed and let the run continue
skipped	the stage was deliberately not run

`tools` records each tool's exit code, wall time and (via `ff_version`) the version string the tool reports; `versions` lists every version recorded in the task, including tools that were probed but not run; `skipped_tools` records steps not run and why (a missing database, not applicable to this isolate). The master table summarises this per isolate in `stages_failed` (`stage:status;...` or `none`), so a row with results is never mistaken for a clean run. The assembly and medaka steps, which have no scientific JSON of their own, emit small status JSONs (`<sample>.assemble.json`, `<sample>.medaka.json`) for the same reason. `|| true` is not used anywhere in the task scripts.

One row per isolate; missing values are `NA` (or `none` for `resistant_classes`). This schema is deliberately organism-agnostic and mirrors the bacterial study's master table so the two One Health analyses are directly comparable.

11.3 11.3 Provenance

Every run ends with a run-level **PROVENANCE** task that writes `pipeline_info/provenance.json` (schema `fungiforge/resources/provenance.schema.json`, checked at write time):

Block	Content	Source
pipeline	name, version, git commit and dirty flag, revision/repository when run from GitHub, script hash	<code>main.nf</code> (head node)
nextflow, run	Nextflow version/build, session id, run name, start time, full command line, launch/work dirs, profile, container engine, stub/resume flags, user, host	Nextflow workflow metadata
params	every effective parameter	Nextflow
containers	process label → container reference that was configured	Nextflow
images	for each reference, what actually ran: a site <code>.sif</code> or cached <code>.img</code> with size and sha256 , or a Docker image id + repo digest	<code>workflow.onComplete</code> on the head node (<code>make_provenance.py</code> images)

Block	Content	Source
databases	the DB_CHECK manifest: each database's key files, <code>.done</code> timestamp and <code>MANIFEST.tsv</code> source	<code>bin/check_databases.py</code>
samples	per isolate, per stage: <code>status</code> , <code>tools</code> (exit, seconds, version), <code>skipped_tools</code> , <code>versions</code> , <code>note</code>	every stage JSON
tool_versions, version_conflicts, stage_summary	one consolidated version per tool across the run (a tool reporting two different versions is listed under <code>version_conflicts</code> — it should be empty), and ok/partial/failed/skipped counts per stage	aggregated

Versions are what each tool prints (e.g. `vsearch v2.29.1_linux_x86_64`); the authoritative package versions of the base image are in `env/base.linux-64.lock`. Hashing the ~30 GB image set takes one to two minutes at the end of the run; `--provenance_hash_images false` records path and size only. On a shared install set `params.image_cache_dir` (site config) to the pre-pulled image cache so cached images resolve; the launcher's `NXF_APPTAINER_CACHEDIR` is used otherwise. Nextflow also writes `pipeline_info/` (execution report, timeline, trace).

12. Layer 2 — the One Health comparative analysis

`analysis/` is a set of **R objective scripts** that consume only the master rows — the same decoupling the bacterial study uses. Run after the pipeline has produced master TSVs:

```
analysis/run_downstream.sh results/ [sample_metadata.csv]
```

Flow: `01_merge_metadata.R` gathers every `04_summary/*.master.tsv` (falling back to `*/14_report/*.master.tsv`) and optionally left-joins richer sample metadata (air volume, health-seeking, ...) by sample id; `02_recover_metadata.R` standardises compartment/env and derives resistance/novelty/BGC logicals into `output/analysis_dataset.csv`, which every objective reads. The eight objectives:

Script	Objective
<code>03_objective1_composition.R</code>	community composition & ID (richness, compartment PERMANOVA, priority pathogens)
<code>04_objective2_resistome.R</code>	antifungal resistome by compartment (azole/echinocandin prevalence, cyp51A TR carriage, Fisher tests)
<code>05_objective3_mobile.R</code>	TE load & mycovirus carriage by compartment
<code>06_objective4_transmission.R</code>	cross-compartment clonality (candidate transmission; SNP/phylo confirm)
<code>07_objective5_risk.R</code>	mixed-effects risk drivers (facility random effect) of resistance/priority carriage
<code>08_objective6_virulence.R</code>	virulence & resistance×virulence convergence (e.g. azole-R <i>A. fumigatus</i> from air)
<code>09_objective7_connectivity.R</code>	shared BGC / mobile-unit connectivity across compartments (BiG-SCAPE + skani)
<code>10_objective8_novelty.R</code>	candidate novel species + novel BGCs / production potential

Conventions match the bacterial study: a shared `_theme.R` with the fixed AIR/HUMAN/SURFACE palette, `set.seed(20260729)`, self-resolving `output/objectiveN + figures/objectiveN`, and `FigureN_*` PNG+PDF at 300 dpi. The outputs are the substrate for the manuscript and seminar deck (built with the `scideck` engine).

Status. The objective scripts are scaffolds: several (connectivity, some risk models) emit summary tables and note where the full analysis (BiG-SCAPE GCF networks, SNP/phylogenetic confirmation) plugs in. They run against real master rows without modification.

13. Interpreting results

Machinery-proven vs needs-real-data. On the `test` fixture and `-stub-run` the DAG *runs* and the JSON contracts *fire*, but the numbers are not biology. Treat as machinery-proven only until a real isolate is processed.

Read every resistance call through its confidence flag. A `provisional_ont_only` substitution or frameshift may be a homopolymer artefact — confirm with a hybrid polish (add Illumina reads and re-run) before reporting it. The **cyp51A promoter TR** is structural and more robust to ONT error, but still confirm the paired coding mutation. **Every resistance call is a genotype** — MIC/AFST is required for a clinical statement.

Read species calls through confidence and method. `high/GCPSR` (ITS + genome ANI concordant) is trustworthy; `medium/low` (ITS below species threshold, or genus-only) needs multi-locus confirmation. A `candidate_novel_species` verdict is a hypothesis for polyphasic follow-up, not a described species.

Know the current placeholders. Decontamination keeps all contigs (BlobTools/organelle extraction pending), efflux over-expression and ploidy are reported as presence/NA (need RNA-seq/read-based tools), and MARDy is not fetched. These are documented honestly in the code and this manual; do not over-read them.

14. Troubleshooting

Symptom	Cause / fix
Missing <code>--samplesheet</code> at launch	provide <code>--samplesheet s.csv</code> with the seven columns
<code>--basecall</code> set but <code>--dorado_model</code> missing	pass <code>--dorado_model <model></code>
ID / resistance / BGC empty, warning about <code>--data_dir</code>	set <code>--data_dir /path/to/fungiforge_db</code> and stage the DBs (§6)
Reference DBs invisible inside a container	the profile binds <code>--data_dir</code> automatically; ensure the path is correct and (Apptainer) in <code>APPTAINER_BINDPATH</code>
Stage 07/11 take many hours locally	linux/amd64 emulation on arm64 hosts — run on a cluster, or accept the wall time; keep <code>--run_interproscan false</code> locally
antiSMASH “command not found” / container exits immediately	the <code>bgc</code> label clears the image entrypoint (<code>--entrypoint=""</code>); keep that in <code>conf/base.config</code>
Medaka/Funannotate crash on <code>getpass.getuser()</code>	the Docker profile sets <code>-e HOME=/tmp -e USER=fungiforge</code> ; keep it
ITSx aborts “over comparison pipeline limit”	it was handed a whole chromosome — <code>extract_rrna_region.py</code> windows the rRNA operon first; ensure barrnap ran
Resistance genes all <code>no_reference</code>	FungAMR references not staged — run <code>bin/fetch_references.sh fungamr</code> (builds <code>reference_proteins.faa</code> from UniProt)
A UNITE/Kraken2/RVDB fetch step logs FAILED	version-sensitive URL 404'd — resolve the current release (recipe in <code>fetch_references.sh</code>) and re-run just that step
Resistance calls all <code>provisional_ont_only</code>	the isolate is ONT-only — add Illumina reads (populate <code>illumina_r1/r2</code>) for a hybrid polish, or run it Illumina-only, for high confidence
Illumina-only isolate not assembling / wrong assembler	check the row has both <code>illumina_r1+illumina_r2</code> and no <code>ont_fastq</code> ; switch <code>--sr_assembler spades\ megahit</code> if SPAdes runs out of memory (MEGAHIT is leaner)
A stage’s container gets pulled despite a skip	ensure you used the <code>--skip_*</code> flag (it emits an empty channel, avoiding the pull)
conda env won’t solve on Apple Silicon	<code>CONDA_SUBDIR=osx-64 mamba env create -f env/<name>.yaml</code> (some builds are linux/osx-64 only)

15. Reproducibility & provenance

Every image is pinned to a versioned tag or digest in `conf/base.config` (§4.3); the base image is built from the explicit conda lock `env/base.linux-64.lock`; the manifest (`manifest.nextflowVersion = '>=23.10'`) pins the engine floor. Databases are verified before every run (`DB_CHECK`, §6.1) and recorded in `--data_dir/MANIFEST.tsv` with their source URL; FungAMR reference building records `reference_build.json`. Every run writes `pipeline_info/provenance.json` (§11.3): commit, Nextflow version, effective parameters, the sha256 or digest of every image that ran, the database manifest, and every tool version per sample and stage. Cite the image digests and database releases recorded there. The One Health analysis pins `set.seed(20260729)`.

16. Extending FungiForge

- **New organism / study:** change the sample sheet metadata and `--busco_lineage`; no code edits.
- **New resistance knowledge:** add rows to `af_resistance_panel.tsv` (gene×species×mechanism×hotspots×known mutations) and rebuild FungAMR references — the caller picks them up.
- **New assembler / polish:** `--assembler {flye,canu,raven}`, `--hybrid {auto,on,off}`.
- **Roadmap (honest):** BlobTools decontam + organelle extraction (Stage 04), read-based ploidy and full secretome (Stage 13), BiG-SCAPE GCF networks and SNP/phylo transmission confirmation (Layer 2), and a real-SRA end-to-end test.

17. Glossary

ONT — Oxford Nanopore long-read sequencing. **Hybrid polish** — correcting an ONT assembly with Illumina short reads (Polypolish). **ITS** — internal transcribed spacer, the fungal barcode. **UNITE** — curated fungal ITS reference (species hypotheses, [SH...FU](#)). **GCPSR** — Genealogical Concordance Phylogenetic Species Recognition. **ANI** — average nucleotide identity. **cyp51A** — the azole target (14- α sterol demethylase / ERG11). **TR34/TR46** — tandem repeats in the cyp51A promoter conferring environmental azole resistance. **FKS1/2** — echinocandin target (glucan synthase), hotspots HS1/HS2. **GOF** / **LoF** — gain-/loss-of-function. **BGC** — biosynthetic gene cluster (NRPS/PKS/terpene/RiPP). **fungiSMASH** — antiSMASH fungal mode. **BUSCO** / **compleasm** — single-copy-ortholog completeness. **MIMAG** — genome-quality reporting standard. **Master table** — the fixed-schema per-isolate TSV that is the Layer-1 $\rightarrow$ Layer-2 handoff.

18. Citation

Cite via `CITATION.cff` in the repository root (Bright Adu; *FungiForge: reproducible fungal genomics from ONT (+ hybrid Illumina)*, v0.1.0, MIT, github.com/aduton1000/fungiforge). FungiForge is the fungal sibling of the forge family — CaptureForge (design), CallForge (targeted germline analysis), MethylForge (methyloomes), and FungiForge (fungal isolate genomics + One Health comparison).